Бази даних

Лекція 7

Тематика лекції

- **SQL, частина 4**
  Підзапити (subqueries) та їх типи
  IN, ANY, ALL, EXISTS
  Корельовані підзапити
  Common Table Expressions (CTE)
  Рекурсивні CTE

Підзапити

- **Підзапит (вкладений запит, subquery, nested query) - запит, вкладений всередину іншого SQL-запиту.**
- **Підзапит завжди береться у дужки.**
- **Результат підзапиту використовується в зовнішньому запиті для фільтрації, порівняння або обчислення.**

Підзапити

```
SELECT student_id, course_id, grade
FROM enrolment
WHERE grade > (
    SELECT AVG(grade) FROM enrolment
);
```

Підзапит повертає одне число (77.31), і зовнішній запит порівнює з ним кожен рядок.
Результат: 99 записів.

Де може стояти підзапит

Частина	Що повертає підзапит	Приклад
WHERE	значення або набір значень для умови	WHERE grade > (SELECT AVG(grade) ...)
SELECT	одне значення для кожного рядка	SELECT ..., (SELECT COUNT(*) ...) AS cnt
FROM	таблицю, з якою працюють як зі звичайною	FROM (SELECT ... GROUP BY ...) AS t
HAVING	значення для умови на групу	HAVING AVG(grade) > (SELECT AVG(grade) ...)

Також: JOIN ... ON, INSERT, UPDATE, DELETE.

Підзапит у SELECT

```
SELECT c.course_name,
       (SELECT COUNT(*) FROM enrolment e
        WHERE e.course_id = c.course_id) AS enrolments
FROM course c
ORDER BY c.course_id;
```

course_name	enrolments
Програмування	35
Математичний аналіз	35
Дискретна математика	35
Алгоритми та структури даних	28
...	...

Кількість записів на кожен курс: одне значення для кожного рядка зовнішнього запиту.

Підзапит у HAVING

```
SELECT c.course_name, ROUND(AVG(e.grade), 2) AS avg_grade
FROM enrolment e
    JOIN course c USING (course_id)
GROUP BY c.course_id, c.course_name
HAVING AVG(e.grade) > (SELECT AVG(grade) FROM enrolment)
ORDER BY avg_grade DESC;
```

course_name	avg_grade
Хмарні технології	86.00
Операційні системи	81.38
Машинне навчання	80.78
Кібербезпека	79.83
Програмування	78.46

Курси, середній бал яких вищий за загальний середній бал (77.31).

Види підзапитів

- **Скалярні - повертають одне значення (1 рядок і 1 колонку).**
- **Колонкові - повертають багато рядків, але 1 колонку.**
- **Рядкові - повертають 1 рядок, але багато колонок.**
- **Табличні - повертають набір з багатьох рядків і колонок.**

Скалярні підзапити

```
SELECT student_id, course_id, grade,
       ROUND((SELECT AVG(grade) FROM enrolment), 2) AS avg_grade
FROM enrolment;
```

student_id	course_id	grade	avg_grade
1	1	61	77.31
1	2	94	77.31
1	3	77	77.31
...	...	...	...

Одне значення, однакове в кожному рядку.

Скалярний: не рівно один рядок

- **Більше одного рядка: помилка.**

```
ERROR:  more than one row returned by a subquery used as an expression
```

- **Нуль рядків: результат NULL, без помилки.**
- **Тому скалярний підзапит варто будувати так, щоб він гарантовано повертав не більше одного рядка: через агрегат (AVG, MAX, COUNT) або умову за первинним ключем.**

Колонкові підзапити

```
-- записи на курси з 6 і більше кредитами
SELECT student_id, course_id, grade
FROM enrolment
WHERE course_id IN (
    SELECT course_id FROM course WHERE credits >= 6
);
```

Підзапит повертає одну колонку з кількома значеннями. Результат: 181 запис.

Рядкові підзапити

```
SELECT student_id, course_id, grade
FROM enrolment
WHERE (course_id, grade) = (
    SELECT course_id, MAX(grade)
    FROM enrolment
    WHERE course_id = 3
    GROUP BY course_id
);
```

student_id	course_id	grade
2	3	99
21	3	99
30	3	99

Рядок підзапиту порівнюється з набором значень у дужках.

Табличні підзапити

```
SELECT p.first_name, p.last_name, ROUND(asg.avg_grade, 2) AS avg_grade
FROM (
    SELECT student_id, AVG(grade) AS avg_grade
    FROM enrolment
    GROUP BY student_id
) AS asg
    JOIN student s USING (student_id)
    JOIN person p USING (person_id)
WHERE asg.avg_grade > 85
ORDER BY asg.avg_grade DESC;
```

first_name	last_name	avg_grade
Мирослава	Гнатюк	97.00
Максим	Новенко	93.20
Інна	Корненко	87.00
Тамара	Юрченко	86.60
Андріана	Волошин	86.00
Марта	Гриценко	85.67

Псевдонім (AS asg) обов’язковий у стандарті SQL і більшості СУБД. PostgreSQL 16+ дозволяє його не писати, але з ним запит читабельніший.

IN, ANY, ALL

Вираз	Значення
x IN (subquery)	x дорівнює хоча б одному значенню. Те саме, що x = ANY (...)
x > ANY (subquery)	x більше хоча б одного значення
x > ALL (subquery)	x більше кожного значення
x NOT IN (subquery)	x не дорівнює жодному значенню. Те саме, що x <> ALL (...)

- x > ALL (порожній набір) дорівнює TRUE для кожного рядка, навіть якщо x є NULL.
- Якщо в наборі є хоча б один NULL, x > ALL (...) ніколи не дорівнює TRUE.

EXISTS

EXISTS (subquery) повертає TRUE, якщо підзапит повернув хоча б один рядок.

```
-- студенти, які не записані на жоден курс
SELECT s.student_id, p.first_name || ' ' || p.last_name AS full_name
FROM student s
    JOIN person p USING (person_id)
WHERE NOT EXISTS (
    SELECT 1
    FROM enrolment e
    WHERE e.student_id = s.student_id
);
```

student_id	full_name
25	Станіслав Левченко
40	Едуард Степаненко

Значення в рядках підзапиту не важливі:
SELECT 1 - це конвенція.

Пастка: NOT IN і NULL

На курсі 7 ("Архітектура комп’ютерів") 6 записів, і в усіх grade є NULL.

```
SELECT COUNT(*)
FROM enrolment
WHERE grade NOT IN (SELECT grade FROM enrolment WHERE course_id = 7);
-- 0

SELECT COUNT(*)
FROM enrolment e
WHERE NOT EXISTS (
    SELECT 1 FROM enrolment e2
    WHERE e2.course_id = 7 AND e2.grade = e.grade
);
-- 251
```

x NOT IN (a, b, NULL) = x <> a AND x <> b AND x <> NULL. Умова ніколи не дорівнює TRUE. Для перевірки "немає відповідного рядка" варто надавати перевагу NOT EXISTS.

Корельовані підзапити

- **Некорельований підзапит не посилається на зовнішній запит. Результат однаковий для всіх рядків, обчислюється один раз.**
- **Корельований підзапит посилається на колонки зовнішнього запиту. Результат свій для кожного рядка, логічно обчислюється для кожного рядка.**
- **Корельованим може бути підзапит будь-якого типу.**

Корельовані підзапити

```
-- оцінка вища за середню на цьому ж курсі
SELECT student_id, course_id, grade
FROM enrolment e
WHERE grade > (
    SELECT AVG(grade)
    FROM enrolment e2
    WHERE e2.course_id = e.course_id
);
```

Умова посилається на рядок зовнішнього запиту e. Результат: 105 записів.

Як виконується підзапит

Підзапит	План (EXPLAIN)	Скільки разів виконується
некорельований скалярний	InitPlan	один раз
корельований скалярний	SubPlan	251 раз, по разу на рядок enrolment
IN, EXISTS	Hash Join (semi-join)	окремо не виконується, перетворюється на JOIN

- "Корельований підзапит завжди повільний" - неточно.
- Повільним може бути корельований скалярний підзапит на великій таблиці.

Альтернатива: віконна функція

```
SELECT student_id, course_id, grade
FROM (
    SELECT e.*, AVG(grade) OVER (PARTITION BY course_id) AS course_avg
    FROM enrolment e
) AS t
WHERE grade > course_avg;
```

Ті самі 105 записів, що й з корельованим підзапитом, але за одне сканування таблиці.

Коли підзапити корисні

- Динамічна фільтрація, коли умова залежить від даних ("оцінки вище середнього").
- Робота з агрегованими даними без кількох окремих запитів.
- Перевірка існування записів (EXISTS).
- Розбиття складної задачі на кроки.
- Менше звернень до бази даних, менше мережевих накладних витрат.
- Увесь оператор SQL бачить дані в одному стані (snapshot), тому результати узгоджені.

Common Table Expressions (CTE)

- **CTE - іменований підзапит, оголошений перед основним запитом за допомогою WITH.**
- **CTE існує лише в межах одного запиту.**
- **CTE допомагають структурувати складні запити: кожен крок має назву.**
- **На один CTE можна посилатися кілька разів.**
- **WITH RECURSIVE дозволяє рекурсивні запити для ієрархічних даних і графів.**

CTE

```
WITH cte_name AS (
    SELECT ...
)
SELECT ...
FROM cte_name;
```

CTE

```
WITH avg_student_grade AS (
    SELECT student_id, AVG(grade) AS avg_grade
    FROM enrolment
    GROUP BY student_id
)
SELECT p.first_name, p.last_name, ROUND(asg.avg_grade, 2) AS avg_grade
FROM avg_student_grade asg
    JOIN student s USING (student_id)
    JOIN person p USING (person_id)
WHERE asg.avg_grade > 85
ORDER BY asg.avg_grade DESC;
```

Той самий запит, що і з табличним підзапитом, і той самий результат (6 рядків).
Проміжний крок має назву, а основний запит читається зверху вниз без вкладень.

Повторне використання CTE

```
WITH student_courses AS (
    SELECT student_id, COUNT(*) AS course_number
    FROM enrolment
    GROUP BY student_id
)
SELECT student_id, course_number
FROM student_courses
WHERE course_number > (SELECT AVG(course_number) FROM student_courses)
ORDER BY course_number DESC, student_id;
```

- CTE використано двічі: як джерело рядків і для середнього (6.61).
- Результат: 21 студент, від 11 курсів до 7 курсів.
- Кілька CTE оголошуються через кому, кожен наступний може посилатися на попередні.

Як виконується CTE

- CTE не обов’язково створює тимчасову таблицю.
- З PostgreSQL 12 CTE, на який посилаються один раз, вбудовується в основний запит так само, як підзапит.
- Якщо на CTE посилаються кілька разів, PostgreSQL обчислює його один раз і зберігає результат.
- Явно керувати цим можна через AS MATERIALIZED і AS NOT MATERIALIZED.
- CTE і підзапит у FROM відрізняються насамперед читабельністю, а не швидкістю.

Рекурсивні CTE

```
WITH RECURSIVE cte_name AS (
    SELECT ...        -- базовий випадок
    UNION ALL
    SELECT ...
    FROM cte_name     -- рекурсивний крок
)
SELECT * FROM cte_name;
```

Як виконується рекурсія

1. Виконується базовий запит. Його рядки йдуть у результат і стають "робочою таблицею".
2. Виконується рекурсивний крок, де cte_name означає лише рядки попереднього кроку.
3. Нові рядки додаються до результату і стають новою робочою таблицею.
4. Коли крок не повертає жодного рядка, рекурсія зупиняється.

Рекурсивні CTE: пререквізити

```
WITH RECURSIVE course_dependencies AS (
    -- базовий випадок: курси без пререквізитів
    SELECT course_id, 1 AS level
    FROM course c
    WHERE NOT EXISTS (
        SELECT 1 FROM course_prerequisite pr WHERE pr.course_id = c.course_id
    )
    UNION ALL
    -- рекурсивний крок
    SELECT pr.course_id, cd.level + 1 AS level
    FROM course_dependencies cd
        JOIN course_prerequisite pr ON pr.prerequisite_course_id = cd.course_id
)
SELECT c.course_name, cd.level
FROM course_dependencies cd
    JOIN course c USING (course_id)
ORDER BY cd.level, c.course_name;
```

Рекурсія покроково

Крок	Нові рядки (course_id)	Пояснення
1 (базовий)	1, 2, 3, 6, 7	Програмування, Математичний аналіз, Дискретна математика, Операційні системи, Архітектура комп’ютерів
2	4, 5, 8, 9, 11, 12	курси, пререквізит яких серед курсів кроку 1
3	8, 10	Машинне навчання (через Алгоритми), Інженерія ПЗ (через ООП)
4	немає	рекурсія зупиняється

Граф, а не дерево

course_name	level
...	...
Машинне навчання	2
...	...
Машинне навчання	3
Інженерія програмного забезпечення	3

- 13 рядків для 12 курсів: "Машинне навчання" має два пререквізити.
- Математичний аналіз -> Машинне навчання: ланцюжок 2.
- Програмування -> Алгоритми -> Машинне навчання: ланцюжок 3.
- Запит повертає рядок для кожного шляху. Для найдовшого ланцюжка: GROUP BY і MAX(level).

Цикли

- Якщо курс A вимагає B, а B вимагає A, запит може ніколи не завершитися.
- Або курси в циклі тихо зникають з результату: від курсу без пререквізитів до них немає шляху.
- PostgreSQL 14+: CYCLE зупиняє рекурсію при повторному відвідуванні.

```
WITH RECURSIVE course_dependencies AS (
    ...
)
CYCLE course_id SET is_cycle USING path
SELECT * FROM course_dependencies;
```

Для старіших версій: обмежити глибину, наприклад WHERE cd.level < 20.

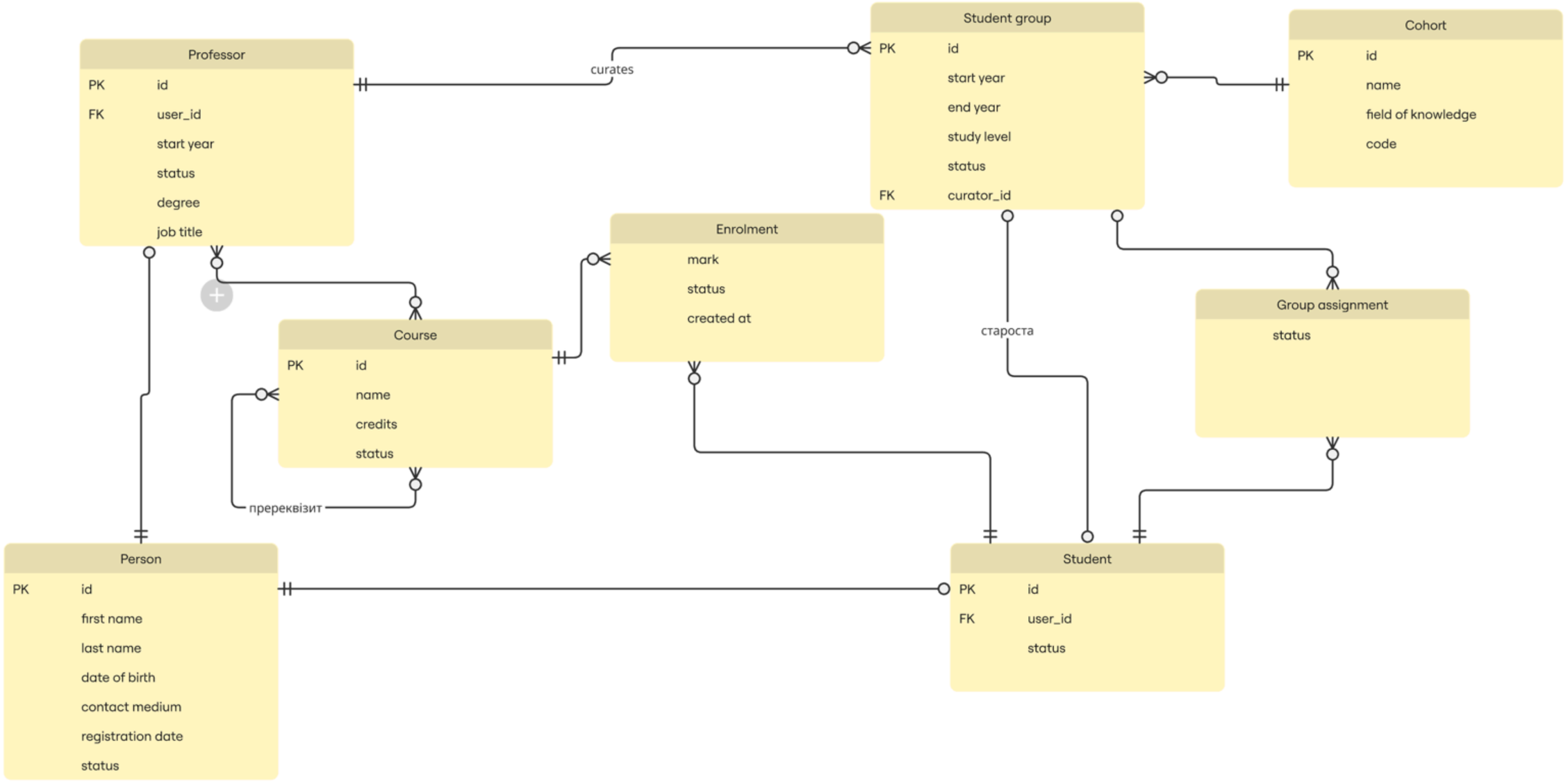
Підзапити і CTE

Ознака	Підзапит	CTE
Розташування	усередині запиту	перед основним запитом
Читабельність	складна при багатьох рівнях вкладення	зверху вниз, кожен крок має назву
Повторне використання	потрібно повторювати текст	можна посилатися кілька разів
Рекурсія	не підтримується	WITH RECURSIVE
Швидкодія	однакова при одноразовому використанні	однакова при одноразовому використанні

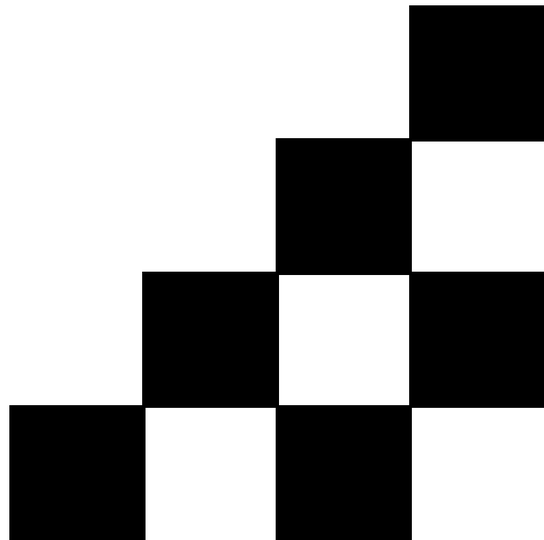
Висновки

- **Підзапит може стояти в WHERE, SELECT, FROM і HAVING. Скалярний повертає не більше одного рядка.**
- **Для перевірки існування варто надавати перевагу EXISTS і NOT EXISTS: NOT IN ламається на NULL.**
- **Корельований скалярний підзапит виконується для кожного рядка. Альтернатива: JOIN з агрегатом або віконна функція.**
- **CTE доречні для запитів з кількома проміжними кроками або повторним використанням результату.**
- **Рекурсивні CTE підходять для ієрархій і графів. Для графа потрібна агрегація, для можливих циклів CYCLE.**
- **З кількох рівнозначних запитів варто надавати перевагу найпростішому.**

Campus


Professor
PK id
FK user_id
start year
status
degree
job title
curates
Student group
PK id
start year
end year
study level
status
FK curator_id
Cohort
PK id
name
field of knowledge
code
Enrolment
mark
status
created at
Course
PK id
name
credits
status
пререквізит
старостa
Group assignment
status
Person
PK id
first name
last name
date of birth
contact medium
registration date
status
Student
PK id
FK user_id
status

Практика
SQL запитів



Запитання